USC Viterbi
School of Engineering

EE 599: Designing and Building Autonomous AI Agents

Units: 2
Term: Fall 2026
Instructor: Arash Saifhashemi
Meeting time, location, office hours, TA, and contact details: TBD

Course description

This course studies the principles behind AI agents: software systems in which a large language model can choose actions, use external tools, and take multiple steps to complete a task.

Students analyze how agents select actions, manage information, and interact reliably with external systems. They examine research evidence, compare system designs, and test their understanding through programming assignments, controlled experiments, and an individual semester project.

Course scope and workload

Expect about four hours of work outside class each week, in addition to the weekly meeting.

Students complete roughly four programming and six reading assignments, an individual midterm examination, and a semester project developed in stages. In-class exercises support this work.

The course examines how agents improve through feedback and experience, including changes to memory, prompts, code, and control logic. Model training and reinforcement learning are introduced to help students understand other approaches to learning and improving agent performance.

Learning objectives

Core concepts

Students will be able to analyze and explain:

- **Model foundations:** How transformers process information and generate responses, why outputs vary, how embeddings support similarity and retrieval, and how model training and reinforcement learning shape model behavior.
- **Agent behavior and control:** Planning, action selection, feedback, application control, and ReAct's cycle of reasoning, acting, and observing results; how agents search and compare candidate solutions, and how additional computation affects quality, cost, and response time.
- **Coordination:** Task decomposition, routing, delegation, handoffs, and the tradeoffs of using multiple agents.
- **Context, state, and memory:** The roles of model context, workflow state, execution checkpoints, and memory across runs; how information is stored, retrieved, corrected, and forgotten.
- **Retrieval and evidence:** How embeddings, similarity, and ranking affect retrieval; whether sources support answer claims and when further investigation is needed.
- **Reliable and secure execution:** Interfaces, validation, process boundaries, timeouts, retries, idempotency, and recovery; how permissions and human approval control actions.

- **Verification and validation:** How tests, constraints, and evidence checks assess outputs and actions; how verifiers can fail; and whether the overall system meets its intended task goals.
- **Evaluation and experimental design:** Repeated trials, uncertainty, scoring methods, baselines, held-out tasks, controlled comparisons, and cost–quality tradeoffs; interpreting `pass@k` (at least one success) and `pass^k` (all k attempts succeed); how errors accumulate across steps and how verification, recovery, and human intervention affect overall task success.
- **Self-improving agents:** How agents use experience, reflection, and feedback to update memory, prompts, code, or control logic; how unreliable tests or model judges can reinforce errors; and how to demonstrate improvement on new tasks.

Practical skills

Students will be able to:

- Build agent loops and stateful workflows using frameworks such as the OpenAI Agents SDK and LangGraph, with clear stopping conditions and failure handling.
- Connect tools through structured interfaces and MCP, with validated arguments and controlled permissions.
- Implement document retrieval and memory across runs, including memory correction and deletion.
- Add validation, human approval, checkpointing, and recovery so an agent can resume interrupted work without performing the same action twice, such as sending a duplicate email.
- Collect execution traces and conduct reproducible evaluations using automated checks, scoring rubrics, cost, and response time.
- Build, test, and explain a complete agent application, including multimodal inputs where appropriate.

Recommended preparation

Students should be comfortable writing and debugging Python, calling HTTP APIs, using Git, and reading basic type annotations. Familiarity with asynchronous programming is helpful.

Prior coursework on transformers or LLMs is not required. The course introduces text generation, the information available in a model's current input, why outputs vary, and why fluency does not guarantee accuracy. Here, **reasoning** means observable multi-step problem solving, such as breaking down a task or using intermediate tool results.

Python features beyond the recommended preparation, including Pydantic, `asyncio`, and decorators, are introduced when an assigned example first uses them.

There are no formal prerequisites.

Software used in the course

Common foundation

- A GitHub account connected to the student's university email address.
- Visual Studio Code and Codex.

Frameworks and tools

Course examples use agent frameworks and tools for MCP integration, retrieval, memory, and evaluation. Specific libraries and setup instructions are provided with each assignment. Early examples use plain Python to make the model–tool loop explicit.

Instructions for obtaining course API access/credits or configuring approved local models (e.g., via Ollama) will be provided on the LMS.

Course format

The class meets once a week for 110 minutes, with a short break. Meetings combine lecture, worked examples, discussion, and programming. The balance of instruction, programming, and project work varies by topic.

Final presentations are prerecorded videos, shown in class on December 4, 2026.

Bring a charged laptop that can run the course environment to every meeting. Selected sessions include coding, and power outlets may be limited. Programming time is for a guided exercise, an assignment, or the semester project.

Selected meetings may feature invited speakers.

The semester project develops from an initial idea through implementation, testing, and refinement.

Assessments and grading

Programming assignments and projects assess students' ability to explain how their systems work, justify design choices, and support conclusions with evaluation results, alongside implementation quality.

Assessment	Weight	Evidence of learning
Four programming assignments	40%	Working code, automated tests, an explanation of design choices, and explanations of failed test cases, their likely causes, and possible improvements; 10% each
Approximately six reading assignments	10%	Short written responses explaining how a paper's method works, whether its results support its claims, its limitations, and when to use it in an agent
Individual midterm examination	20%	Unassisted written exam
Semester project and recorded video presentation	30%	Working code, test results reproducible from the submitted instructions, analysis of security and human approval, final report, recorded demonstration, and individual project assessment
Total	**100%**	

Each reading assignment has the same weight. Together, they account for 10% of the course grade.

Grades and feedback are generally available one week after the due date.

The midterm takes place during the Week 9 class meeting. It covers concepts from Weeks 1–8 and the readings assigned for those weeks.

There is no final examination. The semester project serves as the final assessment.

In-class exercises, draft plans, and peer feedback are not graded on their own, but you can reuse that work in an assignment or the project. Attendance is not graded.

The project is individual. Submit the final project through the LMS by **December 12, 2026**.

A prerecorded project presentation is required for the December 4, 2026 class meeting and counts toward the project grade. Presentation details will be provided with the project instructions.

The project grade includes a brief individual assessment, completed without AI assistance, in which students explain their system's behavior, justify key design decisions, and interpret evaluation results.

Students can discuss project progress and receive feedback during class. The final submission must include an architecture diagram and evaluation results.

Compare your agent with a simpler approach to the same task, and explain the results, failures, and limitations. Detailed evaluation requirements will be provided with the project instructions.

Weekly schedule

The schedule lists weekly topics, selected in-class activities, and deliverables. Changes to the course schedule or assignment deadlines will be announced through the LMS.

Week 1: LLMs, Reasoning Models, and the First Agent Loop

- **Lecture:** Transformers, model training, and text generation; single model calls vs. agents; and the basic agent loop (model -> tool -> result).
- **Student Activity:** Build a plain-Python agent loop using Pydantic for structured outputs. Brainstorm semester project topics.

Week 2: Frameworks, Tools, and Testing

- **Lecture:** Agent framework abstractions: harnesses and runners, enforcing structured outputs, and unit testing non-deterministic systems.
- **Reading assignment 1:** Model and agent foundations; response due before class.
- **Student Activity:** Rebuild the Week 1 loop using the SDK and write tests for a custom tool. *(In-class work on Assignment 1)*

Week 3: Interoperability with Model Context Protocol (MCP)

- **Lecture:** The MCP standard, host/client/server architecture, tools vs. resources, and identifying trust boundaries.
- **Student Activity:** Connect an external MCP server to your agent, or expose your custom tool as an MCP server. Start the threat model.
- **Deliverable:** Assignment 1 due at the start of the week.

Week 4: Prompting Frameworks and Orchestration

- **Lecture:** Reasoning patterns (Chain-of-Thought and ReAct), planning and search, tradeoffs of additional computation, routing, and manager vs. specialist agents.
- **Reading assignment 2:** Reasoning patterns and agent orchestration; response due before class.
- **Student Activity:** Compare provided examples of a fixed workflow and a ReAct agent on the same task. Explain the tradeoffs in reliability, cost, and control. *(In-class work on Assignment 2)*

Week 5: State and Execution in Agent Systems

- **Lecture:** Representing workflows as graphs (nodes and edges), explicit state shared across steps, and basic execution checkpoints.
- **Student Activity:** Build a LangGraph workflow with explicit state and a stop condition; connect an MCP tool.
- **Project preparation:** Discuss your topic and scope in class.

Week 6: Verification, Validation, and Evaluation

- **Lecture:** Verification and validation, pass/fail checks, LLM-as-a-judge rubrics and their limitations, repeated-trial metrics (`pass@k` and `pass^k`), uncertainty, reliability across multiple steps, and cost.
- **Reading assignment 3:** Verification, validation, and evaluation; response due before class.
- **Student Activity:** Write test tasks and scoring rubrics for the agent. Draft the evaluation plan for the semester project.
- **Deliverable:** Assignment 2 due at the start of the week.

Week 7: Retrieval-Augmented Generation (RAG)

- **Lecture:** Retrieval-Augmented Generation (RAG), embeddings and similarity search, and checking claims against retrieved sources.
- **Reading assignment 4:** Retrieval and evidence; response due before class.
- **Student Activity:** Work on Assignment 3: Retrieval-Augmented Generation (RAG).

Week 8: Research Agents and Human-in-the-Loop Control

- **Lecture:** Research-agent workflows, iterative search, synthesis across sources, and evidence assessment; human approval of research plans and review of intermediate results.
- **Deliverable:** Assignment 3 due at the start of the week.

Week 9: Midterm Examination

- **Midterm examination:** Unassisted written exam during the regular class meeting, covering Weeks 1–8 and their assigned readings.

Week 10: Memory, Persistence, and Recovery

- **Lecture:** Context vs. state; long-term memory—what agents write, correct, and forget; execution checkpoints, interruption, state forking, replay, and resuming interrupted runs without doing the same action twice.
- **Reading assignment 5:** Memory and persistence; response due before class.
- **Assignment 4:** Memory and recovery; released this week and due at the start of Week 13.

Week 11: Coding Agents and Learning from Experience

- **Lecture:** Context injection (system rules vs. user prompts), self-improving agents through feedback and reflection on execution traces, the risks of unreliable feedback, adapting tool selection based on past failures, and comparisons with model training and reinforcement learning.
- **Student Activity:** Use a provided agent example and failed trace to predict and test one change on new tasks. Explain the results.

Week 12: Multimodal Agents and Context Management

- **Lecture:** Vision-Language Models (VLMs) for visual UI navigation, managing token costs, and prompt caching.

Week 13: Project Clinic: Evaluation and Failure Analysis

- **Format:** Guided, hands-on work on the semester project.
- **Student Activity:** Compare your agent with a simpler approach to the same task (the baseline), such as one model call or a fixed workflow. Use the same test cases. Investigate one failure, make a targeted change, and rerun the tests.
- **Project preparation:** Record the agent's before-and-after results, cost, and run time, including any new failures, for the final project.
- **Deliverable:** Assignment 4 due at the start of the week.

Week 14: Agent Security and Threat Modeling

- **Lecture:** Prompt injection, unsafe tool use, excessive permissions, and defenses (least privilege, treating output as data).
- **Reading assignment 6:** Agent security and trust; response due before class.
- **Project preparation:** Review your agent for security risks. In the final report, explain the safeguards you implemented and any remaining risks.

Week 15: Emerging Directions and Project Presentations

- **Lecture:** A high-level overview of other frameworks (DSPy, AutoGPT, CrewAI) and future directions in agentic AI.
- **Student Activity:** Show recorded project presentations and discuss the results.
- **Deliverable:** Recorded project presentation due December 4, 2026; included in the project grade.

Final period

- Submit reproducible code, report, evaluation results, and verification and reproducibility note.

Deliverable: Final project due **December 12, 2026**, through the LMS.

Reading assignments

Approximately six reading assignments will examine important research on model foundations, agent architectures, retrieval, memory, evaluation, and reliable operation. In short written responses, students explain how a paper's method works, assess whether its results support its claims, discuss limitations, and connect the findings to agent design. Topics and due weeks are listed in the schedule. Responses are due before the corresponding class meeting; papers and prompts will be posted through the LMS in advance.

There is no required textbook. Assignments use selected papers or excerpts. Software documentation is introduced alongside the examples that use it.

Academic Integrity

The University of Southern California is foremost a learning community committed to fostering successful scholars and researchers dedicated to the pursuit of knowledge and the transmission of ideas. Academic misconduct is contrary to this fundamental mission and includes any act of dishonesty in the submission of academic work (either in draft or final form), as well as cheating, plagiarism, fabrication (e.g., falsifying data), knowingly assisting others in acts of academic dishonesty, and any act that gains or is intended to gain an unfair academic advantage. Students are expected to uphold the highest standards of academic integrity in all coursework. This course follows the expectations for academic integrity as stated in the USC Student Handbook. All students are expected to submit assignments that are original work and prepared specifically for the course/section in this academic term. Students may not submit work written by others or "recycle" work prepared for other courses without obtaining written permission from the instructor(s). Students suspected of academic misconduct will be reported to the Office of Academic Integrity.

Academic dishonesty has a far-reaching impact and is considered a serious offense against the university. Violations will result in a grade penalty, such as a failing grade on the assignment or in the course, and disciplinary action from the university, such as suspension or expulsion. For more information about academic integrity see the Student Handbook, the Office of Academic Integrity's website, and university policies on Research and Scholarship Misconduct. Please ask your instructor if you are unsure what constitutes unauthorized assistance on an exam or assignment or what information requires citation and/or attribution.

Instructors determine whether and how generative AI tools (such as ChatGPT, Copilot, Claude, Gemini, and similar applications) and other external resources (such as English-language translation tools) may be used in this course. Expectations may vary from assignment to assignment. This includes AI tools that record, transcribe, or summarize class sessions, which may require express instructor permission. Review the AI use guidance provided below carefully, and direct any questions to your instructor before submitting work.

Students seeking general guidance may consult the following resources:

- USC Student Handbook
- Generative AI General Policy
- Practical Tips for Using AI in Academic Work
- Office of Academic Integrity

AI policy

AI tools are expected and encouraged for coding, debugging, research, testing, and documentation.

The midterm and individual project assessment are completed without AI assistance.

Each programming assignment and project submission must identify the models used by the agent or its evaluation, explain how the work was tested, and state any known failures or limitations. Students do not need to list coding assistants, individual prompts, or every AI interaction.

Students remain responsible for their work and **must be able to explain their code, reasoning and run, test, and modify submitted code**. Programming assignments may restrict AI use to compare models or measure the effect of AI assistance.

Do not submit private, regulated, proprietary, or personally identifying data to a model or external service. Follow USC's current data guidance and use institutionally approved tools where required.

Ask the instructor if you are unsure whether a particular use of AI is permitted.

Submission and late work

Programming assignments, reading responses, and project submissions are submitted through the course learning-management system.

Late programming assignments and reading responses lose 15% of the available points for each started 24-hour period after the deadline, up to 72 hours. Submissions more than 72 hours late are not accepted. The recorded video presentation and final project must meet their stated deadlines. Documented emergencies, religious observances, and approved accommodations are handled under current USC policy.

A makeup midterm will be offered for justified absences. Contact the instructor as soon as possible to arrange it.

Grading scale

Letter grades are based on the unrounded final percentage.

Grade	Range
A	95–100
A–	90 to <95
B+	87 to <90
B	83 to <87
B–	80 to <83
C+	77 to <80
C	73 to <77
C–	70 to <73
D+	67 to <70
D	63 to <67
D–	60 to <63
F	Below 60

Statement on University Academic and Support Systems

Students and Disability Accommodations

USC welcomes students with disabilities into all of the University's educational programs. The Office of Student Accessibility Services (OSAS) is responsible for the determination of appropriate accommodations for students who encounter disability-related barriers. Once a student has completed the OSAS process (registration, initial appointment, and submitted documentation) and accommodations are determined to be reasonable and appropriate, a Letter of Accommodation (LOA) will be available to generate for each course. The LOA must be given to each course instructor by the student and followed up with a discussion.

This should be done as early in the semester as possible as accommodations are not retroactive. More information can be found at osas.usc.edu. You may contact OSAS at (213) 740-0776 or via email at osasfrontdesk@usc.edu.

Student Financial Aid and Satisfactory Academic Progress

To be eligible for certain kinds of financial aid, students are required to maintain Satisfactory Academic Progress (SAP) toward their degree objectives. Visit the Financial Aid Office webpage for undergraduate- and graduate-level SAP eligibility requirements and the appeals process.

Support Systems

Counseling and Mental Health

Contact: (213) 740-9355, 24/7 on call

Free and confidential mental health treatment for students, including short-term psychotherapy, group counseling, stress fitness workshops, and crisis intervention.

988 Suicide and Crisis Lifeline

Contact: 988 for both calls and text messages, 24/7 on call

The 988 Suicide and Crisis Lifeline (formerly known as the National Suicide Prevention Lifeline) provides free and confidential emotional support to people in suicidal crisis or emotional distress 24 hours a day, 7 days a week, across the United States. The Lifeline consists of a national network of over 200 local crisis centers, combining custom local care and resources with national standards and best practices. The new, shorter phone number makes it easier for people to remember and access mental health crisis services (though the previous 1 (800) 273-8255 number will continue to function indefinitely) and represents a continued commitment to those in crisis.

CARE-SC: Confidential Advocacy, Resources, and Education Support Center

Contact: (213) 740-9355 (WELL), 24/7/365 on call

Confidential advocates, prevention educators, and professional counseling teams work to promote a universal culture of consent, and prevent and respond to gender- and power-based harm. Services available to all USC students at no cost.

Office of Civil Rights Compliance

Contact: (213) 740-5086

Information about how to get help or help someone affected by harassment, discrimination, retaliation on the basis of a protected characteristic, rights of protected classes, reporting options, and additional resources for students, faculty, staff, visitors, and applicants.

USC Campus Support and Intervention

Contact: (213) 740-0411, uscsupport@usc.edu

Assists students and families in resolving complex personal, financial, and academic issues adversely affecting their success as a student.

USC Emergency Information

Latest updates regarding safety, including ways in which instruction will be continued if an officially declared emergency makes travel to campus infeasible.

USC Department of Public Safety

For 24 hour emergency assistance or to report a crime: UPC: (213) 740-4321, HSC: (323)-442-1000.

For 24 hour non-emergency assistance or information: UPC: (213) 740-6000, HSC: 323-442-1200.

Office of the Ombuds

Contact: (213) 821-9556 (UPC) / (323) 442-0382 (HSC)

A safe and confidential place to share your USC-related issues with a University Ombuds who will work with you to explore options or paths to manage your concern.

Occupational Therapy Faculty Practice

Contact: (323) 442-2850 or otfp@med.usc.edu

Lifestyle Redesign services for USC students to support health promoting habits and routines that enhance quality of life and academic performance.

The Office of Sustainability

The Office of Sustainability promotes numerous ways for all Trojans to study, work, and live sustainably at USC. Students can access tools, resources, and opportunities on their website and in the Sustainability Hub in the Student Union Building, Suite 101.

USC Report & Response

USC Report and Response provides a front door to reporting concerns, including an ability to report anonymously, which violate university policies or go against our Unifying Values and helps connect you with the support you need. The website provides a list of confidential resources, supporting offices, and policies that are available to our university community. The Resources page includes information regarding the university's response in the event of immigration enforcement activity on campus or involving a student or employee. Additional questions may be directed to the Office of Professionalism and Ethics (OPE) and ope@usc.edu or in reviewing the Office of Professionalism and Ethics website.

References by core concept

Agent behavior and control

- ReAct: Synergizing Reasoning and Acting in Language Models
- Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Coordination

- Anthropic: Building Effective Agents
- OpenAI Agents SDK: orchestration

Context, state, and memory

- LangGraph overview and saved-state execution
- LangChain memory concepts
- Generative Agents: Interactive Simulacra of Human Behavior
- MemGPT: Towards LLMs as Operating Systems
- Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory
- MemoryAgentBench: Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions

Retrieval and evidence

- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- DPR: Dense Passage Retrieval for Open-Domain Question Answering
- ALCE: Enabling Large Language Models to Generate Text with Citations
- Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection
- STORM: Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models

Reliable and secure execution

- Model Context Protocol architecture
- MCP 2026-07-28 specification release
- OWASP Top 10 for Agentic Applications 2026

Evaluation and experimental design

- OpenAI: evaluate agent workflows
- HumanEval: Evaluating Large Language Models Trained on Code
- τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains
- Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena
- AI Agents That Matter

Self-improving agents

- ReasoningBank: learning from experience
- Reflexion: Language Agents with Verbal Reinforcement Learning